AION: A Closed-Loop, Human-Gated Framework for LLM-Assisted Incident Detection, Root-Cause Analysis, and Validated Remediation

Aditya Bheke
Dept. of Computer Engineering
Ajeenkya D Y Patil
School of Engineering
Pune, India
aditya.bheke@dypic.in

Harshal Bhogawade
Dept. of Computer Engineering
Ajeenkya D Y Patil
School of Engineering
Pune, India
harshal.bhogawade@dypic.in

Malhar Bhoi
Dept. of Computer Engineering
Ajeenkya D Y Patil
School of Engineering
Pune, India
malhar.bhoi@dypic.in

Dr. Yogesh Bahendwar
Dept. of Computer Engineering
Ajeenkya D Y Patil
School of Engineering
Pune, India
yogesh.bahendwar@dypic.in

***Abstract*—Resolving a production incident still requires engineers to search large volumes of logs, trace the failure to the code change that caused it, write and test a fix, and deploy it under time pressure. Prior work automates individual stages of this process—log parsing and anomaly detection, root-cause analysis (RCA), or automated program repair—but the stages are usually studied in isolation, and repair techniques are evaluated on benchmark bugs disconnected from live telemetry, deployment history and release pipelines. We present AION (Autonomous Incident Observation & Navigation), a framework that connects these stages in a single closed loop while keeping a human in control of production. AION deduplicates structured logs into error signatures, detects incidents as signature-level error spikes, ranks suspect commits with an explainable weighted score that combines `git blame`, function scope, dependency overlap and deployment timing, and asks a large language model (LLM) for an evidence-cited diagnosis that is checked against a curated evidence pack. Candidate patches are generated on isolated branches under a code-enforced policy and are accepted only after a deterministic validation pipeline (fail-to-pass regression test, unit tests, staging deployment and replay of the failing requests). Deployment is possible only through an explicit human approval that is bound to the exact validated commit. On four seeded scenarios with known root causes, AION detected every incident with no false positive and ranked the true culprit first in all three faulty releases; with a locally hosted 7B-parameter model, its LLM diagnosis named the culprit in two of three cases, and every incorrect AI patch was rejected by validation before reaching the approval gate.**

***Index Terms*—AIOps, incident management, log analysis, root-cause analysis, large language models, automated program repair, human-in-the-loop, continuous deployment**

I. Introduction

Modern software services produce continuous streams of logs, alerts and deployment events. When a release introduces a fault, on-call engineers must find the relevant error among thousands of log lines, decide which recent change caused it, write and test a fix, and push that fix through the deployment pipeline. Empirical studies of cloud incident management report that this process remains expensive and dependent on experienced engineers, even in organisations with mature tooling [1], [2].

Consider a typical example, which we use as a running case study in Section VII. An online store deploys release v1.4.0, which contains a campaign feature that makes the coupon lookup return `None` for expired codes. A few hours later, customers who enter an old coupon receive HTTP 500 errors, and the logs fill with `TypeError: 'NoneType' object is not subscriptable`. To resolve this, an engineer must (i) notice that this error is new and not background noise, (ii) find which of the commits in the release touched the failing code, (iii) understand why it fails, (iv) write a fix and a test, (v) verify that the fix works and breaks nothing else, and (vi) deploy it. Each step is tedious, and each is a place where an automated assistant could help—provided it does not make production worse.

Research in AIOps has made substantial progress on each stage separately. Log parsers such as Drain [3] and LILAC [4] convert raw logs into templates; DeepLog [5], LogAnomaly [6], NeuralLog [7] and BERT-based detectors [8]–[10] detect anomalous log behaviour. RCA techniques localise faulty services [11]–[13], and recent work shows that LLMs can recommend root causes and mitigations for real cloud incidents [14]–[16]. In parallel, LLM-based automated program repair (APR) and software engineering agents can generate and test code fixes [17]–[20].

Two gaps remain. First, these stages are rarely connected: detection and RCA systems stop at a diagnosis, while repair systems start from a bug report or a failing test and are evaluated on benchmarks such as Defects4J or SWE-bench [21] rather than on live production signals, deployment history and release pipelines. Recent surveys identify this fragmentation, together with hallucination and unreliable evaluation, as open problems for LLM-based AIOps [22], [23]. Second, letting an

LLM change production code is unsafe without guarantees: a plausible but wrong fix can make an incident worse, and work on LLM-based remediation explicitly calls for safety checks [24].

AION addresses both gaps with a closed-loop workflow in which AI may *investigate, analyse, propose and validate*, but only a human may *approve* a production change. This paper makes the following contributions:

- An end-to-end architecture that connects log-based detection, change correlation, LLM diagnosis, patch generation, deterministic validation and deployment, with the human approval gate enforced structurally by a finite-state machine and bound to a commit hash (Section IV).
- An explainable change-correlation score that combines line- and function-level `git blame`, file and dependency overlap and deployment timing, extending the classical idea of fix-inducing changes [25] to online incident triage (Section V-C).
- An evidence-pack and grounding scheme that requires every LLM claim to cite a verifiable evidence item and caps confidence when claims cannot be verified (Section V-D).
- A policy-constrained generate–validate–repair loop with fail-to-pass checking, staging request replay and a revert fallback (Sections V-E–V-F).
- A scenario-based evaluation with known answers that measures each stage separately and documents how the evaluation exposed and fixed four weaknesses of the initial design (Section VIII).

The remainder of the paper is organised as follows. Section II reviews related work. Section III formalises the problem and the safety requirements. Section IV presents the architecture and Section V the methods used in each stage. Section VI describes the implementation, Section VII walks through a complete incident, and Section VIII reports the evaluation. Sections IX and X discuss lessons, limitations and threats to validity, and Section XI concludes.

II. Related Work

A. Log Parsing and Anomaly Detection

Surveys of automated log analysis cover the full pipeline from logging practice through parsing, anomaly detection, failure prediction and diagnosis [26]; He *et al.* reviewed 158 publications spanning more than two decades. Public datasets such as Loghub [27] have enabled systematic benchmarking across distributed systems, supercomputers, operating systems and server applications.

Log parsing turns free-text messages into structured events. Drain [3] parses logs online with a fixed-depth tree that groups messages into templates. LILAC [4] uses LLMs with in-context learning and an adaptive parsing cache, which improves template accuracy while reducing the number of LLM queries by orders of magnitude.

Anomaly detection identifies abnormal log behaviour. DeepLog [5] models normal log-key sequences with an LSTM and flags deviations, and LogAnomaly [6] detects both sequential and quantitative anomalies. Because parsing errors propagate into detectors, NeuralLog [7] embeds raw messages directly and classifies them with a Transformer. LogBERT [8] and LAnoBERT [9] apply self-supervised BERT models trained on normal logs, and a recent industrial study deploys a LogBERT model locally to satisfy confidentiality requirements [10]. A systematic review of LLM-based log analysis [23] highlights log drift, long-tail events, hallucination and the context, cost and latency limits of LLMs.

These methods establish *that* something is wrong; they do not identify the responsible change or repair it. AION deliberately uses a lightweight and deterministic detector (template masking plus spike detection), because its contribution lies in what happens after detection, and because a deterministic detector is easy to explain and audit.

B. AIOps and Incident Management

Notaro *et al.* [2] organise AIOps failure-management methods into prevention, detection, diagnosis and remediation, and Zhang *et al.* [22] survey how LLMs are changing these tasks, noting that the limits of LLMs in AIOps and the evaluation of LLM-based methods are not yet well understood. At Microsoft, Chen *et al.* [1] report on two years of incident-management practice and present an AIOps framework for incident management. Ahmed *et al.* [14] evaluate GPT-3.x models on more than 40,000 incidents for recommending root causes and mitigation steps from the incident title and summary, and include a human evaluation with incident owners. These studies show that LLMs can assist engineers, but they work from incident text rather than from code and deployment history, and they do not produce or verify code changes.

C. Root-Cause Analysis

Service-level RCA methods such as MicroRCA [11] and AutoMAP [12] localise faulty microservices from metrics and dependency graphs; Soldani and Brogi [13] survey anomaly detection and RCA in multi-service applications. Wittkopp *et al.* [28] progress from anomaly detection towards automated log labelling and RCA. LLM-based RCA systems for cloud incidents [15] and LLM agents for RCA [16] show that diagnosis improves when the model is given relevant context. Integrating LLMs with cloud-native observability for automated RCA and remediation [24] highlights the risks of hallucinated root causes, telemetry volumes that exceed the model's context, and remediation without safety checks. AION adopts the lesson that context quality matters, but replaces free-form retrieval with a bounded, citable evidence pack, and links the diagnosis to a concrete, testable code change.

D. Change Analysis

The SZZ approach of Śliwerski *et al.* [25] links version history to bug reports to identify fix-inducing changes after the fact, and observes that such changes show distinct patterns in size and timing. AION applies a related intuition online: when an error spike follows a deployment, the commits that touched the failing lines, functions or their dependencies, and

TABLE I
STAGE COVERAGE OF REPRESENTATIVE WORK (✓ COVERED, ○ PARTIAL, – NOT COVERED)

Work	Det.	Corr.	RCA	Repair	Valid.	Gate
DeepLog, NeuralLog [5], [7]	✓	–	–	–	–	–
LogBERT, LAnoBERT [8], [9]	✓	–	–	–	–	–
MicroRCA, AutoMAP [11], [12]	○	–	✓	–	–	–
LLM incident RCA [14], [15]	–	–	✓	–	–	–
SZZ [25]	–	✓	–	–	–	–
AlphaRepair, LLM-APR [17], [30]	–	–	–	✓	○	–
RepairAgent, AutoCodeRover [18], [19]	–	–	○	✓	✓	–
AION (this work)	✓	✓	✓	✓	✓	✓

Det.: detection from production logs; Corr.: correlation with commits and deployments; Valid.: validation before release; Gate: enforced human approval for production.

that shipped in that deployment, are the strongest suspects. Unlike SZZ, AION must rank suspects *before* a fix exists, and must explain its ranking to a human reviewer.

E. Automated Program Repair and LLM Agents

Le Goues *et al.* [29] review search-, semantics- and learning-based APR. AlphaRepair [30] showed that pre-trained code models can repair bugs zero-shot, without training on bug-fix pairs, and Xia *et al.* [17] found across nine models and five datasets that larger pre-trained models outperform earlier APR tools; Zhang *et al.* [31] review the rapidly growing LLM-based APR literature. RepairAgent [18] is an autonomous agent that gathers information, proposes fixes and validates them with tools, repairing 164 Defects4J bugs at a cost of roughly 270K tokens per bug. AutoCodeRover [19] combines program-structure-aware code search with LLMs to resolve GitHub issues, and SWE-agent [20] designs agent–computer interfaces for software engineering. SWE-bench [21] evaluates such systems on 2,294 real GitHub issues from 12 Python repositories. Kulsum *et al.* [32] show that chain-of-thought reasoning and patch-validation feedback improve LLM-generated vulnerability fixes.

These systems start from an issue or a failing test and end with a candidate patch. None of them observes production, correlates deployments, reproduces failing production traffic or gates deployment on human approval.

F. Grounding and Human-in-the-Loop Control

Retrieval-augmented generation [33] conditions a generator on retrieved passages to make outputs more specific and factual. AION follows the same principle but with a curated, structured context in which every item has an identifier that the model must cite; this allows claims to be verified mechanically. Human oversight is a recurring theme in incident management [1], [14]; AION makes it a structural property of the system rather than a recommendation.

G. Positioning

Table I summarises the stages covered by representative work. AION's novelty is not a better individual stage but the safe, auditable composition of all stages, with deterministic checks around every AI output.

TABLE II
FUNCTIONAL REQUIREMENTS

ID	Requirement	Addressed in
R1	Ingest structured logs from files and over HTTP	Sec. V-A
R2	Collapse repeated errors into signatures	Sec. V-A
R3	Open an incident when an error rate spikes	Sec. V-B
R4	Rank suspect commits with explanations	Sec. V-C
R5	Diagnose the root cause with cited evidence	Sec. V-D
R6	Generate a candidate fix and a regression test	Sec. V-E
R7	Validate the fix against tests and real traffic	Sec. V-F
R8	Deploy only after approval; verify recovery	Sec. V-G
R9	Work without an LLM, and with a local model	Sec. V-D

III. PROBLEM FORMULATION

A. Definitions

A monitored *service* has a Git repository with a production branch and a sequence of *deployments* $D = \langle d_1, \ldots, d_n \rangle$, where each d_j records a version, the deployed commit and a timestamp. The service emits a stream of structured log events L. An *incident* I is a set of error events that share a signature and whose rate departs from its baseline. Let C be the set of commits that reached production in recent deployments. The incident-resolution task is to produce, for each incident:

1) a ranked list of suspect commits $\langle c_1, c_2, \ldots \rangle \subseteq C$ with an explanation for each rank;
2) a diagnosis R that states the root cause and cites evidence;
3) a candidate patch P (a commit on an isolated branch) together with a validation report $V(P)$; and
4) a production deployment of P, performed only after a human decision A.

B. Safety Properties

Because candidate patches may be produced by an LLM, AION treats every AI output as untrusted and must guarantee the following properties regardless of model behaviour:

- **S1 (Human authority):** No change reaches production without an explicit human approval.
- **S2 (Approval integrity):** The deployed commit is exactly the commit that was validated and approved.
- **S3 (Validated candidates):** Only patches that pass deterministic validation are offered for approval.
- **S4 (Bounded edits):** A patch cannot modify tests, CI configuration or dependencies, and cannot exceed a size limit.
- **S5 (Traceability):** Every automated and human action is recorded and attributable.

C. Functional Requirements

Table II lists the functional requirements derived from the task and the module that addresses each one.

TABLE III
BACKEND COMPONENTS

Package	Responsibility
`logs`	file-tail collector, HTTP ingest, normalisation, templates, signatures, spike detection
`gitops`	Git wrapper (history, blame, worktrees, revert); commit–deployment correlation
`ai`	provider-neutral LLM interface, evidence pack, RCA, grounding, patch prompts
`remediation`	worktree per attempt, edit application, patch policy, revert patches
`validation`	local CI runner: syntax, fail-to-pass, tests, staging, replay
`deploy`	approval-checked deployment and production verification
`pipeline`	background worker and orchestrator; lifecycle state machine
`api`	REST endpoints used by the dashboard and log shippers

IV. SYSTEM ARCHITECTURE

Fig. 1 shows the closed loop. A monitored production service emits structured JSON logs. AION, implemented as a modular monolith, processes them through eight stages; stages 1–6 run automatically in a background pipeline, stage 7 requires a human, and stage 8 runs only after approval. Every transition is appended to an audit trail.

A. Design Principles

- **AI proposes, humans dispose.** The incident lifecycle is a finite-state machine (Fig. 2); no transition leads from AWAITING_APPROVAL to DEPLOYING except through APPROVED, which only the human approval API can enter (property S1).
- **Deterministic checks around every AI output.** LLM diagnoses are grounded against evidence; LLM patches pass a code-enforced policy and a validation pipeline whose commands are defined by the operator, never by the model (S3, S4).
- **Explainability.** Every suspect score, grounding warning and validation step is stored with a human-readable reason.
- **Graceful degradation.** Without an LLM, a clearly labelled deterministic analyser reports the top-ranked suspect and proposes a `git revert` (R9).
- **Replaceable infrastructure.** Each infrastructure stand-in—the validation runner, the deployer, the LLM provider—sits behind a narrow interface so that it can be replaced by a production equivalent without changing the rest of the pipeline.

B. Components

AION is organised as a modular monolith: a single backend process whose packages have clear boundaries and communicate through function calls and the database. Table III lists the packages. A single background worker executes pipeline jobs, so each incident is processed sequentially and its state changes are easy to follow.

TABLE IV
MAIN PERSISTENT ENTITIES

Entity	Key attributes
Service	repository path, production branch, health path, test and run commands
LogEvent	timestamp, level, message, template, signature, exception, frames, request
LogSignature	signature, template, count, first and last occurrence
Incident	title, status, severity, signature, event count, first and last seen
Deployment	version, commit SHA, time, status, related incident
CommitSuspect	commit SHA, author, message, files, score, reasons
RCAReport	root cause, observations, inferences, suspected commit, confidence, grounding warnings, evidence pack
PatchProposal	attempt, strategy, edits, diff, branch, base and commit SHA, status
ValidationRun	per-step status and output
Approval	approver, decision, comment, commit SHA
AuditEvent	timestamp, actor, action, details

TABLE V
TEMPLATE MASKING RULES (APPLIED IN ORDER)

#	Pattern	Token
1	UUID	<UUID>
2	HTTP(S) URL	<URL>
3	e-mail address	<EMAIL>
4	IPv4 address (optional port)	<IP>
5	hexadecimal literal or long hex identifier	<HEX>
6	ISO-8601 timestamp	<TS>
7	integer or decimal number	<NUM>

C. Data Model

All state is persisted in a relational database. Table IV summarises the main entities. Every analysis artefact belongs to exactly one incident, so a reviewer can reconstruct the full history of an incident—including what the LLM was shown—from the database alone.

V. METHODOLOGY

A. Log Normalisation and Signatures

Each log record l is normalised into an event with timestamp, level, message, request context and parsed stack frames. A template $\tau(l)$ is obtained by applying the masking rules of Table V in order, so that more specific patterns are replaced before generic numbers; for example, *"Order 1042 failed after 3.2ms"* becomes *"Order <NUM> failed after <NUM>ms"*. For error events, the signature is a hash

$$s(l) = h\big(\text{type}(l),\ \text{frames}(l),\ \tau(l)\big), \tag{1}$$

where $\text{frames}(l)$ are the innermost application frames (file and function, deliberately *without* line numbers so that a signature survives unrelated edits above the failing line). Signatures are stored with occurrence counts and first and last occurrence times, which turns thousands of lines into a handful of rows.

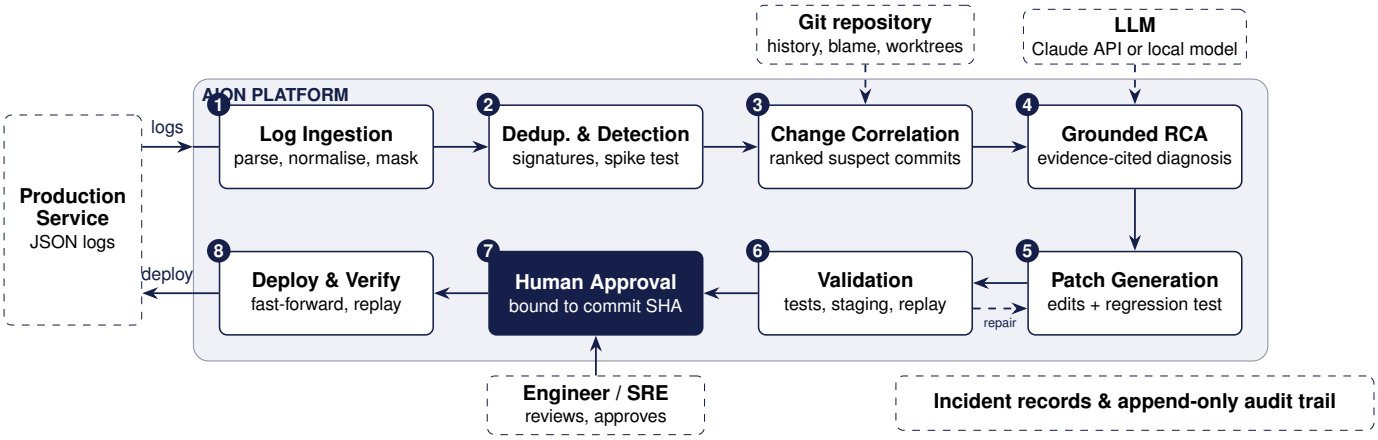


Fig. 1. AION closed-loop architecture. Stages 1–6 run automatically; production deployment (stage 8) is reachable only through an explicit human approval (stage 7). Failed validation loops back to patch generation a bounded number of times.

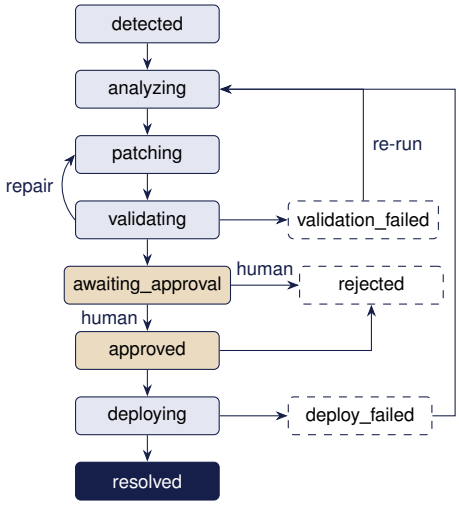


Fig. 2. Incident lifecycle. Shaded states require a human decision; there is no automated edge into APPROVED.

B. Incident Detection

Let $c_t(s)$ be the number of events with signature s in the current window of length W, and $b(s)$ the average number of occurrences per window over the preceding baseline period B. An incident is opened when

$$c_t(s) \geq m \quad \wedge \quad c_t(s) \geq \rho \cdot b(s), \tag{2}$$

with defaults $W = 60\,\text{s}$, $B = 30\,\text{min}$, $m = 5$ and $\rho = 3$. A brand-new error ($b(s) = 0$) fires once it reaches m, while long-standing low-rate noise does not. Further events with the same signature attach to the open incident. Event time (log timestamps) rather than wall-clock time is used, so replayed logs behave identically to live traffic. Events older than the closure of a previous incident with the same signature are ignored, so late log lines cannot reopen a fixed problem, whereas a genuine recurrence after closure opens a new incident. Algorithm 1 summarises the procedure.

Algorithm 1 Signature-level spike detection

Input: new error signatures Σ in an ingested batch, time t
Output: set of incidents opened

$k \leftarrow \max(B/W, 1)$; $\quad O \leftarrow \emptyset$
for $s \in \Sigma$ **do**
  if an open incident with signature s exists **then** attach events; **continue**
  end if
  $t_0 \leftarrow \max(t - W,\ \textsc{LastClosure}(s))$
  $c \leftarrow \textsc{Count}(s, t_0, t)$
  $b \leftarrow \textsc{Count}(s, t - W - B, t - W)/k$
  if $c \geq m \wedge c \geq \rho\, b$ **then**
    $O \leftarrow O \cup \{\textsc{OpenIncident}(s, c, b)\}$ ▷ state DETECTED
  end if
end for
return O

C. Change Correlation

For each commit c in the recent history of the service, AION computes signals $f_k(c)$ and combines them into an explainable suspicion score

$$S(c) = \min\left(1,\ \sum_k w_k\, f_k(c)\right) \cdot \lambda(c), \qquad c^* = \arg\max_c S(c), \tag{3}$$

with the signals and weights of Table VI. Blame signals are computed at the *deployed* revision, function ranges are found with Python's `ast` module, and signals from deeper stack frames are damped by a factor of 0.5. Each signal counts once per commit (its strongest instance), so a commit that touches many frames cannot inflate its score. Commits already running in an earlier deployment before the errors appeared receive $\lambda(c) = 0.5$; otherwise $\lambda(c) = 1$. Ties are broken by recency. $S(c)$ is a ranking heuristic, not a probability; the LLM later reasons over the top candidates and their diffs.

The dependency-overlap signal deserves comment. A fault is not always located in the code that crashes: in one of our scenarios, a commit adds a product record with a differently named price field, and existing, correct pricing code raises a `KeyError` when it reads that record. `git blame` on the

TABLE VI
CHANGE-CORRELATION SIGNALS AND WEIGHTS

Signal	Condition	w_k
Blame (line)	commit last modified the failing line	0.40
Blame (function)	commit modified the enclosing function	0.20
Deployment timing	commit shipped in the deployment preceding the errors	0.20
File overlap	commit touched a file in the stack trace	0.15
Dependency overlap	commit touched a module imported by a file in the trace	0.15
Keywords	error identifiers appear in the commit message	0.05

TABLE VII
EVIDENCE PACK ITEMS

ID	Content
`LOG-n`	deduplicated log clusters around the incident, with counts and a classification (incident, new, pre-existing noise)
`TRACE-1`	a representative stack trace and sample failing requests
`DEPLOY-n`	deployments preceding the incident
`COMMIT-x`	top suspect commits with scores, reasons and diffs
`CODE-n`	source of the functions in the stack trace at the deployed revision
`INC-n`	similar past incidents and how they were fixed

failing line points at the old pricing commit. The dependency signal credits commits that changed a module *imported by* a file in the stack trace, which captures causes that lie in data or helpers rather than on the stack.

D. Evidence Pack and Grounded Root-Cause Analysis

Instead of placing raw logs in the prompt or retrieving free text [33], AION assembles a bounded *evidence pack* E whose items carry stable identifiers (Table VII). The pack is small enough to fit the context window of a local 7B model (12k tokens in our experiments), and its items are selected for relevance rather than retrieved by similarity.

The LLM must return structured JSON that separates *observations*, each citing an evidence ID, from *inferences*, each citing supporting evidence, together with affected files, a suspected commit, a remediation plan and a confidence value. A grounding step then checks the response deterministically:

- observations citing unknown evidence IDs are removed;
- inferences that cite unknown evidence, or none, are flagged;
- affected files must exist in the repository;
- the suspected commit must be one of the candidate commits; a disagreement with the top-ranked suspect is reported to the reviewer;
- any unverifiable claim caps the confidence at 0.60, and a diagnosis with no verifiable observation is capped at 0.30.

The exact evidence pack is stored with the report, so a reviewer sees what the model saw. The provider layer is model-neutral: AION supports the Claude API with structured outputs and any OpenAI-compatible endpoint, including locally hosted models, which keeps sensitive logs on premises. Without a configured model, a deterministic analyser produces a labelled diagnosis from the top-ranked suspect and the failing frame.

TABLE VIII
PATCH POLICY ENFORCED IN CODE

Rule
Paths must be repository-relative, without `..`, drive letters or absolute paths
Edited files must already exist in the repository
Tests, CI configuration, Dockerfiles, dependency manifests and `.git` are protected
New test files must match a dedicated name pattern (`tests/test_aion_*.py`)
At most 5 files and 200 changed lines per patch

Algorithm 2 Generate–validate–repair with revert fallback

Input: incident I, evidence pack E, diagnosis R, suspects C, attempt budget N
Output: validated candidate P, or escalation to an engineer

$fb \leftarrow \emptyset$
for $i = 1$ **to** N **do**
  $P \leftarrow \text{LLMPATCH}(E, R, fb)$ ▷ edits + regression test
  $P \leftarrow \text{REPAIRINDENTATION}(P)$
  if $\neg\,\text{POLICY}(P)$ **then** $fb \leftarrow$ violation; **continue**
  end if
  $V \leftarrow \text{VALIDATE}(P)$ ▷ Sec. V-F
  if V.passed **then return** P ▷ to approval gate
  end if
  $fb \leftarrow V$.failures
end for
$P \leftarrow \text{REVERT}(\arg\max_{c \in C} S(c))$
if $\text{VALIDATE}(P)$.passed **then return** P
end if
return $\text{ESCALATE}(I)$ ▷ state VALIDATION_FAILED

E. Policy-Constrained Patch Generation

Each patch attempt runs in its own Git worktree and branch, so concurrent or failed attempts never touch the production checkout. The LLM returns search/replace edits plus a regression test that should reproduce the incident. Edits are applied literally; if the search text differs from the source only in indentation, the edit is repaired automatically and the repair is recorded. A code-enforced policy (Table VIII), independent of the prompt, rejects patches that violate safety property S4—so a model that ignores its instructions still cannot, for example, delete a failing test to make the suite pass.

Algorithm 2 shows the bounded generate–validate–repair loop: policy violations and validation failures are fed back to the model for a limited number of attempts (two by default), after which AION falls back to reverting the top-ranked suspect commit. Both kinds of candidate go through the same validation and the same human gate. A revert is a conservative remedy—it removes the new feature together with the fault—so the reviewer is shown which strategy produced the candidate.

TABLE IX
VALIDATION PIPELINE (A BLOCKING FAILURE STOPS THE PIPELINE)

Step	Check
syntax	every changed source file compiles
fail2pass	new regression test fails on the old code (reproduces the incident)
tests	the service's full unit-test suite passes on the patched code
staging	patched service starts from its worktree and passes its health check
replay	failing production requests now succeed; baseline requests keep their status

F. Deterministic Validation

A patch P is accepted only if

$$V(P) = \text{syntax} \wedge \text{fail2pass} \wedge \text{tests} \wedge \text{staging} \wedge \text{replay}, \quad (4)$$

with the steps of Table IX. The fail-to-pass step runs the new regression test against the *unpatched* code, where it should fail, proving that it reproduces the incident; for revert candidates, which add no test, this step is skipped. The replay step re-sends the production requests that failed during the incident (expecting no 5xx response) together with a sample of previously successful requests (expecting an unchanged status), so a patch that "fixes" the crash by breaking normal behaviour is rejected. All commands come from the operator's service registration and run without a shell and with timeouts (180 s for tests by default). This mirrors the finding that validation feedback is essential for trustworthy LLM fixes [32], but executes it against reproduced production traffic rather than a benchmark test suite.

G. Human Approval, Deployment and Audit

An engineer reviews the evidence, diagnosis, diff and validation report in a web dashboard and approves or rejects the candidate. The approval records the approver, a comment and the commit SHA; deployment is a separate action that re-checks that the approved SHA equals the validated SHA (property S2):

$$\text{deploy}(P) \iff V(P) = 1 \ \wedge \ A = \text{approve} \wedge \ \text{sha}(A) = \text{sha}(P). \quad (5)$$

An approval may also be withdrawn before deployment. Deployment performs pre-flight checks and a GitOps-style fast-forward of the production branch, followed by verification that the service reports the approved commit and that the previously failing requests now succeed. The incident is marked resolved only after verification; a failed verification moves it to DEPLOY_FAILED and allows re-investigation. All actions by the pipeline and by humans are written to an append-only audit table (property S5).

TABLE X
MAIN REST ENDPOINTS

Endpoint	Purpose
`POST /api/services`	register a monitored service
`POST /api/ingest/{service}`	ingest a batch of log records
`POST /api/deployments`	record a deployment
`GET /api/incidents[/{id}]`	list incidents / full detail
`POST .../{id}/approve`	approve the validated candidate
`POST .../{id}/reject`	reject or withdraw approval
`POST .../{id}/deploy`	deploy the approved commit
`POST .../{id}/rerun`	re-run the investigation
`GET /api/audit`	read the audit trail

TABLE XI
MAIN CONFIGURATION PARAMETERS AND DEFAULTS

Parameter	Meaning	Default
`DETECTION_WINDOW`	window W	60 s
`DETECTION_BASELINE`	baseline B	1800 s
`DETECTION_MIN_COUNT`	threshold m	5
`DETECTION_SPIKE_RATIO`	ratio ρ	3.0
`MAX_PATCH_ATTEMPTS`	AI attempts N	2
`REVERT_FALLBACK`	revert after failures	on
`TEST_TIMEOUT`	test timeout	180 s
`LLM_PROVIDER`	model provider	auto

VI. IMPLEMENTATION

AION is implemented in Python with FastAPI for the REST interface, SQLAlchemy and SQLite in write-ahead-logging mode for persistence, and the Git command-line tool for history, blame, worktrees and reverts. The dashboard is a React single-page application built with Vite and served by the backend; it polls the API for live updates. The LLM layer supports the Claude API through the official SDK, with structured outputs, and OpenAI-compatible servers such as Ollama for local models. Table X lists the main endpoints and Table XI the main configuration parameters.

The dashboard has four views: an incident list; an incident detail page with a live pipeline stepper, log clusters, the evidence-cited diagnosis, ranked suspects, a diff viewer, the validation report and the approval panel; an audit log; and a system page showing services, the configured analyser and worker health. A realistic demonstration service—an orders API with a synthetic multi-commit, multi-deployment Git history and a GitOps-style controller that reloads the running service when the production branch moves—is monitored end to end.

The code base is covered by 44 automated tests, including unit tests for normalisation, detection, correlation, the patch policy and the lifecycle, and complete end-to-end workflow tests. In a live demonstration on a laptop, an injected fault reached the approval gate about 4 s after the first errors with the deterministic analyser, and approval plus deployment reached a verified, resolved state in about 2 s.

TABLE XII
RECORDED SUSPECTS FOR THE EXPIRED-COUPON INCIDENT

Commit	$S(c)$	Recorded reasons
Support seasonal campaigns: coupon expiry and maximum discount	1.00	blame on failing line; modified `apply_discount()`; shipped in v1.4.0; file in trace; changed imported module `coupons.py`; keyword "discount"
Add coupon support to order totals	0.375	modified `apply_discount()`; blame on calling line; files in trace; imported module; already in production before v1.4.0 ($\times$0.5)
Add request duration to access logs	0.35	shipped in v1.4.0; file in trace

TABLE XIII
EVALUATION SCENARIOS (KNOWN ANSWERS)

Scenario	Fault	What it tests
expired-coupon	`TypeError` on expired coupon	culprit changed the crashing line
supplier-feed	`KeyError: 'price'`	culprit changed only data; blame points at old code
empty-cart-average	`ZeroDivisionError`	latent condition becomes fatal after a new feature
healthy-release	none	false-positive check with background noise

VII. CASE STUDY: AN EXPIRED-COUPON INCIDENT

We now follow the running example of Section I through the pipeline, using the recorded output of the evaluation harness.

Setup. The orders service has five base commits (initial API, pricing with GST, coupons, payments, documentation). Two further commits are then deployed together as release v1.4.0: a campaign feature by one developer that adds coupon expiry and a maximum discount and makes `get_coupon()` return `None` for unknown or expired codes, and an unrelated change by another developer that adds request durations to the access logs. Traffic then includes orders that use an expired coupon.

Detection. The harness generated 44 log events. The new `TypeError` signature reached the threshold in its first window with an empty baseline, and exactly one incident was opened, titled *"TypeError: 'NoneType' object is not subscriptable"*.

Correlation. Table XII shows the top three suspects with their recorded reasons. The campaign commit collects every signal ($0.40+0.20+0.20+0.15+0.15+0.05 = 1.15$, capped at 1.0). The older commit that introduced coupon support modified the enclosing function and the calling line in a deeper frame ($0.40 \cdot 0.5 + 0.20 + 0.15 + 0.15 + 0.05 = 0.75$), but it was already in production before the errors began, so its score is halved to 0.375. The logging change shipped in the same release and touched a file in the trace, giving 0.35. The ranking is therefore unambiguous, and each number can be traced to a human-readable reason.

Diagnosis. The deterministic analyser reported the failing location (`app/pricing.py`, line 17, in `apply_discount()`) and the campaign commit as the most recent change to it, with confidence 0.8. The local LLM, given the evidence pack, explained that the error occurs because the `coupon` parameter is `None` where a dictionary is expected, and named the same commit with confidence 0.9; the grounding step raised no warnings.

Remediation and validation. The local model's two patch attempts guarded one field of the `None` coupon but still read another, so its own regression test crashed and both attempts were rejected. The revert fallback then reverted the campaign commit on an isolated branch; the candidate compiled, passed the unit tests, started in staging and answered the replayed requests correctly, and the incident moved to AWAITING_APPROVAL. At this point an engineer sees the diagnosis, the ranked suspects with reasons, the revert diff and the validation report, and decides whether to deploy the revert or to write a forward fix.

VIII. EVALUATION

A. Research Questions

- RQ1 Does AION detect seeded incidents without false positives?
- RQ2 Does change correlation rank the true culprit first?
- RQ3 Does the LLM diagnosis identify the culprit, and do AI patches pass validation?
- RQ4 Does the safety design stop incorrect patches before the approval gate?

B. Setup

We built four scenarios on the same five-commit base history of the demonstration service, each with a known answer (Table XIII). The *supplier-feed* scenario is deliberately hard: the culprit commit changes only *data*, so `git blame` on the failing line points at older, innocent code. The *healthy-release* scenario ships a harmless change with background noise and checks for false positives. For each scenario an evaluation harness creates a fresh workspace and database, builds the Git history, executes the real service code to generate normal traffic and trigger requests, ingests the resulting logs through the API and runs the real pipeline up to the approval gate. Production deployment is excluded because it always requires a human.

We compare two modes: *heuristic* (no LLM: correlation-ranked diagnosis and revert) and *LLM*, using qwen2.5-coder:7b (Q4_K_M quantisation) served locally by Ollama on a laptop GPU with 6 GB of memory, a 12k-token context and temperature 0.1, with at most two AI patch attempts followed by the revert fallback. We record detection correctness, the rank of the true culprit, whether the diagnosis names it, the sequence of patch attempts and their validation results, the final status, wall-clock time and token usage.

C. Results

Table XIV reports the final results and Table XV the per-scenario detail of the LLM run.

TABLE XIV
FINAL EVALUATION RESULTS

Metric	Heuristic	Local LLM (7B)
Detection correct (incl. no false positive)	4/4	4/4
True culprit ranked first by correlation	3/3	3/3
Diagnosis names true culprit	3/3	2/3
AI patches passing validation	–	0/6
Reached approval gate or correctly silent	4/4	3/4
Incorrect patches reaching the gate	0	0
Mean time per incident	≈5 s	≈100 s
Mean LLM tokens in / out per incident	–	≈13.5k / 1.2k

TABLE XV
PER-SCENARIO RESULTS OF THE LOCAL-LLM RUN

Scenario	Events	Conf.	Time	Tokens in/out	Patch sequence → status
empty-cart-average	39	0.9	100.1 s	13,236 / 1,183	AI×, AI×, revert✓ → gate
expired-coupon	44	0.9	109.2 s	15,775 / 1,222	AI×, AI×, revert✓ → gate
supplier-feed	39	0.8	97.5 s	11,397 / 1,280	AI×, AI× → validation failed
healthy-release	31	–	–	–	no incident (correct)

TABLE XVI
EVALUATION-DRIVEN IMPROVEMENTS (GATE = REACHED APPROVAL GATE OR CORRECTLY SILENT; RCA = DIAGNOSIS NAMES CULPRIT)

Run	Change since previous run	Heur. gate/RCA	LLM gate/RCA
1	initial design	3/4 · 2/3	1/4 · 2/3
2	dependency signal, scoped keywords	4/4 · 3/3	–
3	indentation-only edit repair	–	1/4 · 2/3
4	three AI attempts instead of two	–	1/4 · 2/3
5	revert fallback after failed AI patches	4/4 · 3/3	3/4 · 2/3

TABLE XVII
WHY THE 7B MODEL'S PATCHES FAILED

Scenario	Model behaviour	Caught by
empty-cart-average	correct fix, but regression test used a non-empty order	its own regression test
expired-coupon	guarded one field of `None`, still read another	its own regression test
supplier-feed	priced the product at 0 instead of reading the new field	test-suite step

RQ1 (detection). Both modes detected all three incidents with the correct exception type and correctly stayed silent on the healthy release; each scenario produced between 31 and 44 log events and exactly one incident where one was expected.

RQ2 (correlation). The true culprit was ranked first in all three faulty scenarios. In supplier-feed it was found through the dependency-overlap and deployment-timing signals ($0.20 + 0.15 + 0.05 = 0.40$), tying with the old pricing commit that `git blame` points at ($0.40{+}0.20{+}0.15{+}0.05 = 0.80$, halved to 0.40 because it was already in production); the tie was broken by recency. We report this narrow margin explicitly: the ranking is correct but fragile in this scenario.

RQ3 (diagnosis and repair). The local LLM explained all three failures in plain language and named the culprit commit in two of three cases (confidence 0.9); in supplier-feed it described the symptom accurately but named no commit. None of the six AI-generated patches passed validation.

RQ4 (safety). Every incorrect AI patch was rejected by deterministic checks, and the revert fallback produced validated candidates for two of the three incidents; the third was escalated as VALIDATION_FAILED. No incorrect patch reached the approval gate in any run.

D. Evaluation-Driven Improvements

The evaluation was run five times while the system was improved, and each run exposed a concrete weakness (Table XVI). In the first run, `git blame` ranked the supplier-feed culprit third and every LLM patch failed the syntax check because of indentation errors. Adding the dependency-overlap signal and restricting the keyword signal to the specific error moved the culprit to rank one in all scenarios. Indentation-only repair made AI patches compile and produced regression tests that correctly failed on the buggy code, although the fixes themselves remained semantically wrong. A third AI attempt did not help the 7B model, whereas a revert fallback rescued two incidents. All changes were generic and covered by unit tests, rather than tuned to individual scenarios.

E. Failure Analysis of AI Patches

Table XVII lists why the local model's patches failed. In empty-cart-average the fix itself was correct, but the model's regression test exercised a non-empty order and therefore did not reproduce the incident. In expired-coupon the guard was incomplete. In supplier-feed the model tried to "fix" the crash by pricing a product without a price field at zero instead of reading the supplier's field; in the recorded final attempt, its own regression test asserted a subtotal of 0.00, the patched code returned 3499.0, and the test-suite step therefore failed (1 failed, 9 passed). Each failure was caught by a different deterministic check, which supports the decision to treat every AI output as untrusted until validated.

F. Cost and Latency

The deterministic path processes an incident in about 5 s, dominated by Git operations, the test suite and the staging start-up. With the local 7B model the end-to-end time rises to about 100 s per incident, dominated by generation on a 6 GB laptop GPU, with roughly 11–16k input and 1.2–1.3k output tokens per incident across one diagnosis and two patch attempts. Since the deterministic stages always run first, an engineer receives detection, ranked suspects and a heuristic diagnosis within seconds, regardless of the speed of the model.

IX. DISCUSSION

Diagnosis versus repair. With a small local model, diagnosis was considerably more reliable than repair: the model explained every failure but produced no validated fix. This matches observations that repair quality grows with model size [17] and that agents need validation feedback [18], [32]. Because AION's provider layer is model-neutral, the same harness can measure stronger hosted models with a configuration change. A practical consequence is that even a weak model adds value in AION's design: its explanation helps the

reviewer, while the deterministic stages supply the ranking and the safe fallback.

Safety by construction. The approval guarantee does not depend on model behaviour: it is a property of the state machine, the SHA binding and the operator-defined validation commands. The evaluation shows the value of this separation: every wrong patch was stopped automatically, and every remaining decision went to a human. This contrasts with fully autonomous repair agents, whose safety depends on the quality of the generated patch and the available tests.

Explainable correlation. The weighted score is simple, but every component is a human-readable reason, which helps reviewers and also guides the LLM. The supplier-feed case shows that blame alone is insufficient when the culprit changes data rather than code; the dependency-overlap signal addressed this generically, although only narrowly.

Evaluation as a design tool. A single successful demonstration would have hidden all four weaknesses listed in Table XVI. Scenario-based evaluation with known answers, run after every change, turned anecdotes into measurable regressions and improvements, and we recommend it for any LLM-assisted operations tool.

Practical deployment. The infrastructure stand-ins used in this work—a local validation runner, local staging and a fast-forward deployer—sit behind narrow interfaces. In an organisation they would be replaced by a CI system, a container-based staging environment and a GitOps controller, and the incident report could be exported as a ticket. Because a local model is supported, organisations that cannot send logs to an external provider can still use the LLM stages.

Ethical and security considerations. Logs may contain personal data or secrets; sending them to an external LLM is a data-protection decision, which AION leaves to the operator by supporting local models. AI-generated code is executed during validation, so it must run with the least privilege available; the patch policy prevents the most obvious manipulations, but a container sandbox is preferable. Finally, a human approver must not become a rubber stamp: AION therefore shows the evidence, the reasons for every score and the validation output, not only a recommendation.

X. Threats to Validity

Internal validity. The scenarios were authored by the same team that built the system, and prompt guidance was adjusted after observing failures. To limit overfitting, every change was generic (indentation repair, dependency signal, keyword scoping, revert fallback) and covered by a unit test, and the scenarios were designed to include an adversarial case for `git blame`.

External validity. The evaluation is small—four scenarios on one Python service—and each configuration was run once, so LLM variance is not measured; the results characterise behaviour and failure modes rather than general accuracy. Results on a 7B quantised model do not predict the behaviour of larger hosted models in either direction.

Construct validity. "Diagnosis names the culprit" measures whether the suspected commit is correct, not the quality of the explanation; "reached the approval gate" measures automated success, not whether an engineer would accept the candidate.

Limitations of the current system. Detection uses logs only, with fixed thresholds and hand-tuned weights, and only Python stack traces are parsed. Staging and production are local processes, request replay covers idempotent requests only, and AI-generated code is validated in a separate process rather than a container sandbox. The current version has no authentication or role-based approvers and no automatic rollback.

XI. Conclusion and Future Work

AION shows that log-based detection, change correlation, LLM diagnosis, patch generation, validation and deployment can be connected in a single closed loop without giving AI the authority to change production. Safety follows from the architecture—a lifecycle state machine with no automated path to approval, an approval bound to the validated commit, a code-enforced patch policy and operator-defined validation—rather than from the quality of the model. In our evaluation, detection and correlation were accurate and deterministic, a 7B local model was useful for diagnosis but not for repair, and deterministic validation together with a commit-bound human gate stopped every incorrect patch.

Future work falls into three groups. *Evaluation:* repeated runs with stronger hosted models, a larger benchmark of seeded incidents across several services, and studies with practising engineers. *Coverage:* metrics- and trace-based detection, adaptive thresholds and learned correlation weights, support for further languages, and retrieval over runbooks and post-mortems. *Operations:* integration with real CI/CD systems and container-based staging, sandboxed validation, secret and personal-data redaction before prompting, role-based and two-person approval, pull-request-based fixes, canary releases and automatic rollback on failed verification.

Acknowledgment

The authors thank the Department of Computer Engineering, Ajeenkya D Y Patil School of Engineering, Pune, for its support of this work.

References

[1] Z. Chen *et al.*, "Towards intelligent incident management: Why we need it and how we make it," in *Proc. 28th ACM Joint Meeting on European Software Engineering Conf. and Symp. Foundations of Software Engineering (ESEC/FSE)*, 2020, pp. 1487–1497.

[2] P. Notaro, J. Cardoso, and M. Gerndt, "A survey of AIOps methods for failure management," *ACM Transactions on Intelligent Systems and Technology*, vol. 12, no. 6, pp. 81:1–81:45, 2021.

[3] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, "Drain: An online log parsing approach with fixed depth tree," in *Proc. IEEE Int. Conf. Web Services (ICWS)*, 2017, pp. 33–40.

[4] Z. Jiang *et al.*, "LILAC: Log parsing using LLMs with adaptive parsing cache," in *Proc. ACM Int. Conf. Foundations of Software Engineering (FSE)*, 2024.

[5] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly detection and diagnosis from system logs through deep learning," in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2017, pp. 1285–1298.

[6] W. Meng *et al.*, "LogAnomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs," in *Proc. 28th Int. Joint Conf. Artificial Intelligence (IJCAI)*, 2019, pp. 4739–4745.

[7] V.-H. Le and H. Zhang, "Log-based anomaly detection without log parsing," in *Proc. 36th IEEE/ACM Int. Conf. Automated Software Engineering (ASE)*, 2021, pp. 492–504.

[8] H. Guo, S. Yuan, and X. Wu, "LogBERT: Log anomaly detection via BERT," arXiv:2103.04475, 2021.

[9] Y. Lee, J. Kim, and P. Kang, "LAnoBERT: System log anomaly detection based on BERT masked language model," arXiv:2111.09564, 2023.

[10] M. De la Cruz Cabello, T. P. Sales, and M. R. Machado, "Log anomaly detection in AIOps: A real-world implementation using large language models," *Systems and Soft Computing*, vol. 8, p. 200475, 2026.

[11] L. Wu, J. Tordsson, E. Elmroth, and O. Kao, "MicroRCA: Root cause localization of performance issues in microservices," in *Proc. IEEE/IFIP Network Operations and Management Symp. (NOMS)*, 2020.

[12] M. Ma, J. Xu, Y. Wang, P. Chen, Z. Zhang, and P. Wang, "AutoMAP: Diagnose your microservice-based web applications automatically," in *Proc. The Web Conference (WWW)*, 2020, pp. 246–258.

[13] J. Soldani and A. Brogi, "Anomaly detection and failure root cause analysis in (micro)service-based cloud applications: A survey," *ACM Computing Surveys*, vol. 55, no. 3, pp. 1–39, 2022.

[14] T. Ahmed *et al.*, "Recommending root-cause and mitigation steps for cloud incidents using large language models," in *Proc. IEEE/ACM 45th Int. Conf. Software Engineering (ICSE)*, 2023.

[15] Y. Chen *et al.*, "Automatic root cause analysis via large language models for cloud incidents," in *Proc. 19th European Conf. Computer Systems (EuroSys)*, 2024.

[16] D. Roy *et al.*, "Exploring LLM-based agents for root cause analysis," in *Companion Proc. 32nd ACM Int. Conf. Foundations of Software Engineering (FSE)*, 2024.

[17] C. S. Xia, Y. Wei, and L. Zhang, "Automated program repair in the era of large pre-trained language models," in *Proc. IEEE/ACM 45th Int. Conf. Software Engineering (ICSE)*, 2023.

[18] I. Bouzenia, P. Devanbu, and M. Pradel, "RepairAgent: An autonomous, LLM-based agent for program repair," in *Proc. IEEE/ACM 47th Int. Conf. Software Engineering (ICSE)*, 2025, arXiv:2403.17134.

[19] Y. Zhang, H. Ruan, Z. Fan, and A. Roychoudhury, "AutoCodeRover: Autonomous program improvement," in *Proc. 33rd ACM SIGSOFT Int. Symp. Software Testing and Analysis (ISSTA)*, 2024.

[20] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024.

[21] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" in *Proc. 12th Int. Conf. Learning Representations (ICLR)*, 2024.

[22] L. Zhang, T. Jia, M. Jia *et al.*, "A survey of AIOps in the era of large language models," *ACM Computing Surveys*, 2026.

[23] Z. Ma, J. Yang, and T.-H. Chen, "LLM4Log: A systematic review of large language model-based log analysis," arXiv:2604.16359, 2026.

[24] "Integrating large language models with cloud-native observability for automated root cause analysis and remediation," in *Proc. 3rd Int. Conf. Artificial Intelligence, Systems and Network Security*, 2025.

[25] J. Śliwerski, T. Zimmermann, and A. Zeller, "When do changes induce fixes?" in *Proc. 2nd Int. Workshop on Mining Software Repositories (MSR)*, 2005, pp. 24–28.

[26] S. He, P. He, Z. Chen, T. Yang, Y. Su, and M. R. Lyu, "A survey on automated log analysis for reliability engineering," *ACM Computing Surveys*, vol. 54, no. 6, pp. 130:1–130:37, 2021.

[27] J. Zhu, S. He, P. He, J. Liu, and M. R. Lyu, "Loghub: A large collection of system log datasets for AI-driven log analytics," in *Proc. IEEE 34th Int. Symp. Software Reliability Engineering (ISSRE)*, 2023.

[28] T. Wittkopp, A. Acker, and O. Kao, "Progressing from anomaly detection to automated log labeling and pioneering root cause analysis," in *Proc. AIOps Workshop, IEEE Int. Conf. Data Mining (ICDM)*, 2023, arXiv:2312.14748.

[29] C. Le Goues, M. Pradel, and A. Roychoudhury, "Automated program repair," *Communications of the ACM*, vol. 62, no. 12, pp. 56–65, 2019.

[30] C. S. Xia and L. Zhang, "Less training, more repairing please: Revisiting automated program repair via zero-shot learning," in *Proc. 30th ACM ESEC/FSE*, 2022.

[31] Q. Zhang *et al.*, "A systematic literature review on large language models for automated program repair," arXiv:2405.01466, 2024.

[32] U. Kulsum, H. Zhu, B. Xu, and M. d'Amorim, "A case study of LLM for automated vulnerability repair: Assessing impact of reasoning and patch validation feedback," in *Proc. 1st ACM Int. Conf. AI-Powered Software (AIware)*, 2024.

[33] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.